

tests\test_utils_calvindate.py

```
1  """Tests for calvincTools.utils.calvindate module."""
2  import pytest
3  from datetime import datetime, date, timedelta
4  from calvincTools.utils.calvindate import calvindate, IsDateString
5
6
7  class TestCalvindateConstruction:
8      """Test calvindate construction methods."""
9
10     def test_construction_no_args(self):
11         """Test construction with no arguments returns today."""
12         cd = calvindate()
13         today = datetime.today()
14         assert cd.year == today.year
15         assert cd.month == today.month
16         assert cd.day == today.day
17
18     def test_construction_year_month_day(self):
19         """Test construction with year, month, day."""
20         cd = calvindate(2024, 11, 15)
21         assert cd.year == 2024
22         assert cd.month == 11
23         assert cd.day == 15
24
25     def test_construction_month_day(self):
26         """Test construction with month and day (current year)."""
27         cd = calvindate(12, 25)
28         current_year = date.today().year
29         assert cd.year == current_year
30         assert cd.month == 12
31         assert cd.day == 25
32
33     def test_construction_from_date_object(self):
34         """Test construction from date object."""
35         d = date(2024, 6, 15)
36         cd = calvindate(d)
37         assert cd.year == 2024
38         assert cd.month == 6
39         assert cd.day == 15
40
41     def test_construction_from_datetime_object(self):
42         """Test construction from datetime object."""
43         dt = datetime(2024, 6, 15, 14, 30, 45)
44         cd = calvindate(dt)
45         assert cd.year == 2024
46         assert cd.month == 6
47         assert cd.day == 15
48         assert cd.hour == 14
49         assert cd.minute == 30
```

```

50         assert cd.second == 45
51
52     def test_construction_from_string(self):
53         """Test construction from date string."""
54         cd = calvindate("2024-11-15")
55         assert cd.year == 2024
56         assert cd.month == 11
57         assert cd.day == 15
58
59     def test_construction_from_string_various_formats(self):
60         """Test construction from various date string formats."""
61         formats = [
62             "11/15/2024",
63             "November 15, 2024",
64             "15 Nov 2024",
65             "2024-11-15"
66         ]
67         for fmt in formats:
68             cd = calvindate(fmt)
69             assert cd.year == 2024
70             assert cd.month == 11
71             assert cd.day == 15
72
73     def test_construction_with_time(self):
74         """Test construction with date and time components."""
75         cd = calvindate(2024, 11, 15, 14, 30, 45)
76         assert cd.year == 2024
77         assert cd.month == 11
78         assert cd.day == 15
79         assert cd.hour == 14
80         assert cd.minute == 30
81         assert cd.second == 45
82
83     def test_construction_with_microseconds(self):
84         """Test construction with microseconds."""
85         cd = calvindate(2024, 11, 15, 14, 30, 45, 123456)
86         assert cd.microsecond == 123456
87
88
89     class TestCalvindateMethods:
90         """Test calvindate methods."""
91
92     def test_value(self):
93         """Test value method returns self."""
94         cd = calvindate(2024, 11, 15)
95         assert cd.value() == cd
96
97     def test_as_datetime(self):
98         """Test as_datetime conversion."""
99         cd = calvindate(2024, 11, 15, 10, 30, 45)
100        dt = cd.as_datetime()

```

```
101     assert isinstance(dt, datetime)
102     assert dt.year == 2024
103     assert dt.month == 11
104     assert dt.day == 15
105     assert dt.hour == 10
106     assert dt.minute == 30
107
108     def test_daysfrom_positive(self):
109         """Test daysfrom with positive delta."""
110         cd = calvindate(2024, 11, 15)
111         future = cd.daysfrom(5)
112         assert future.day == 20
113         assert future.month == 11
114         assert future.year == 2024
115
116     def test_daysfrom_negative(self):
117         """Test daysfrom with negative delta."""
118         cd = calvindate(2024, 11, 15)
119         past = cd.daysfrom(-5)
120         assert past.day == 10
121         assert past.month == 11
122         assert past.year == 2024
123
124     def test_daysfrom_cross_month(self):
125         """Test daysfrom crossing month boundary."""
126         cd = calvindate(2024, 11, 28)
127         future = cd.daysfrom(5)
128         assert future.day == 3
129         assert future.month == 12
130         assert future.year == 2024
131
132     def test_tomorrow(self):
133         """Test tomorrow method."""
134         cd = calvindate(2024, 11, 15)
135         tomorrow = cd.tomorrow()
136         assert tomorrow.day == 16
137         assert tomorrow.month == 11
138         assert tomorrow.year == 2024
139
140     def test_yesterday(self):
141         """Test yesterday method."""
142         cd = calvindate(2024, 11, 15)
143         yesterday = cd.yesterday()
144         assert yesterday.day == 14
145         assert yesterday.month == 11
146         assert yesterday.year == 2024
147
148     def test_tomorrow_month_boundary(self):
149         """Test tomorrow at month boundary."""
150         cd = calvindate(2024, 11, 30)
151         tomorrow = cd.tomorrow()
```

```

152         assert tomorrow.day == 1
153         assert tomorrow.month == 12
154
155     def test_yesterday_month_boundary(self):
156         """Test yesterday at month boundary."""
157         cd = calvinder(2024, 12, 1)
158         yesterday = cd.yesterday()
159         assert yesterday.day == 30
160         assert yesterday.month == 11
161
162
163     class TestCalvinderNextWorkday:
164         """Test nextWorkdayAfter method."""
165
166     def test_next_workday_from_friday(self):
167         """Test next workday from Friday is Monday."""
168         # December 6, 2024 is a Friday
169         cd = calvinder(2024, 12, 6)
170         next_wd = cd.nextWorkdayAfter()
171         # Should be Monday, December 9
172         assert next_wd.day == 9
173         assert next_wd.month == 12
174         assert next_wd.weekday() == 0 # Monday
175
176     def test_next_workday_from_monday(self):
177         """Test next workday from Monday is Tuesday."""
178         # December 2, 2024 is a Monday
179         cd = calvinder(2024, 12, 2)
180         next_wd = cd.nextWorkdayAfter()
181         # Should be Tuesday, December 3
182         assert next_wd.day == 3
183         assert next_wd.month == 12
184         assert next_wd.weekday() == 1 # Tuesday
185
186     def test_next_workday_include_after_date(self):
187         """Test next workday with include_afterdate=True."""
188         # December 2, 2024 is a Monday (workday)
189         cd = calvinder(2024, 12, 2)
190         next_wd = cd.nextWorkdayAfter(include_afterdate=True)
191         # Should be the same day since Monday is a workday
192         assert next_wd.day == 2
193         assert next_wd.month == 12
194
195
196     class TestIsDateString:
197         """Test IsDateString function."""
198
199     def test_is_date_string_valid(self):
200         """Test valid date strings."""
201         valid_dates = [
202             "2024-11-15",

```

```
203         "11/15/2024",
204         "November 15, 2024",
205         "15 Nov 2024"
206     ]
207     for date_str in valid_dates:
208         assert IsDateString(date_str) is True
209
210     def test_is_date_string_invalid(self):
211         """Test invalid date strings."""
212         invalid_dates = [
213             "not a date",
214             "hello world",
215             "12345",
216             ""
217         ]
218         for date_str in invalid_dates:
219             assert IsDateString(date_str) is False
220
221     def test_is_date_string_edge_cases(self):
222         """Test edge cases."""
223         # Ambiguous but parseable
224         assert IsDateString("1/1/1") is True
225         # Invalid date
226         assert IsDateString("2024-13-45") is False
227
```